



Transactional Lakehouse with Apache Iceberg

Reliable and Scalable Data Architecture

Madhurima Saxena & Raghav Nagarajan

Western US Architecture Series

Context & Scale

Built from scratch to power leadership dashboards across 3500+ global annotators and 80+ upstream data sources. No existing pipelines to reuse — had to get it right from day one.

The customer needed data fast — in dashboards, in data marts — but the team was primarily BI engineers, dependent on the core team for infrastructure before they could even start building. We needed an architecture that could deliver quickly and be self-maintainable by non-data-engineers.

TECH STACK

Airflow — Orchestration

Python & PySpark — Processing

Apache Iceberg — Table format

AWS (Glue, Athena, SQS, SNS) — Cloud services

Trino — Query engine

Tableau — Visualization

CI/CD, GitHub — DevOps

DataHub — Data discovery

SCALE TODAY

150+ pipelines (~1,500 runs/week)

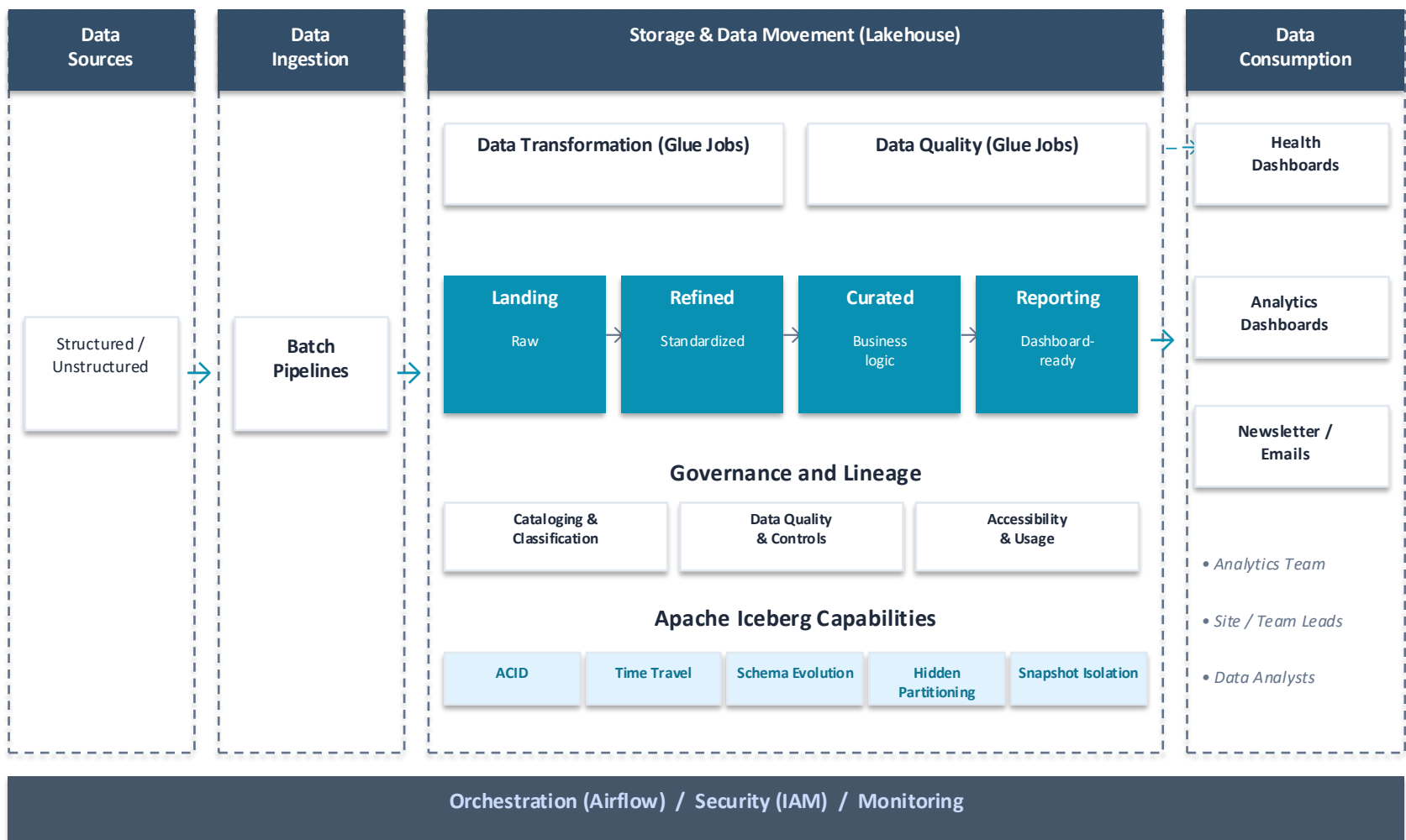
400+ tables

20+ dashboards

80 upstream data sources

~3,500 annotator customers

Architecture Overview



Platform Capabilities

Unified Data Catalog

Centralized metadata and documentation for easy discovery, transparency, and reuse across teams.

Data Quality Controls

Automated jobs validate and improve data quality across stages — raw to refined to curated.

Medallion Architecture

Landing, Refined, Curated, Reporting — each zone scales independently with clear team ownership.

BI-Optimized Modeled Data

Structured for reporting and analytics, driving fast adoption among BI and reporting teams.

Single Source of Truth

Curated Zone is the authoritative version for consistent reporting and insight generation.

Access Control by Design

Landing zone access-restricted for security; BI users work with curated/reporting zones tailored to analysis.

Problem 1: Unreliable Data

With legacy pipelines - Nobody trusted the dashboards

PROBLEM

1. Late-arriving data meant different teams saw different numbers
2. **No way to know what changed or when** — debugging was guesswork
3. Stakeholder confidence in reporting was dropping

APPROACH & OUTCOME

1. **Immutable snapshots** — every write is tracked; time travel for deterministic debugging
2. **Partition-aware MERGE** — scoped by business keys & snapshot metadata; only affected files rewritten
3. **Late data handling** — compare snapshots, find diffs, recompute only impacted partitions

DESIGN CHOICE: WHY ICEBERG

Our data model was constantly evolving — schemas changed and partitions weren't stable.
We needed a format that could adapt without breaking pipelines or requiring expensive rewrites.

Iceberg manages partitions and schema at the metadata level, allowing us to evolve both safely.
This made it a strong fit for a fast-moving, multi-source environment.

Problem 2: Fragile, Unscalable Pipelines

With legacy pipelines - Schema changes broke everything, and full refreshes couldn't keep up

PROBLEM

1. A column rename upstream would break downstream pipelines
2. **Ingestion and reporting tightly coupled** — one failure cascaded everywhere
3. Full-refresh pipelines grew slower and more expensive as data scaled

APPROACH & OUTCOME

1. **Zero-downtime schema evolution** — column adds, renames, drops without breaking consumers
2. **Only process what changed** — snapshot diffs identify changed rows, replay downstream

STRUCTURE: Medallion Architecture



The Refined layer separates raw ingestion from business logic — source-side changes don't cascade into curated models. Reporting serves as the semantic layer: aggregated, dashboard-ready.

SCALE: DAG Strategy

Source-level DAGs at Landing — one DAG per source system for simplicity

Table-level DAGs at Curated — one task/DAG per table for fine-grained retry, parallelism, and dependency management

This separation lets each layer scale independently



Learnings & Architecture Evolution

Evolution 1: Proliferating scripts in the Refined Zone (~Bronze)

Abstract to scale

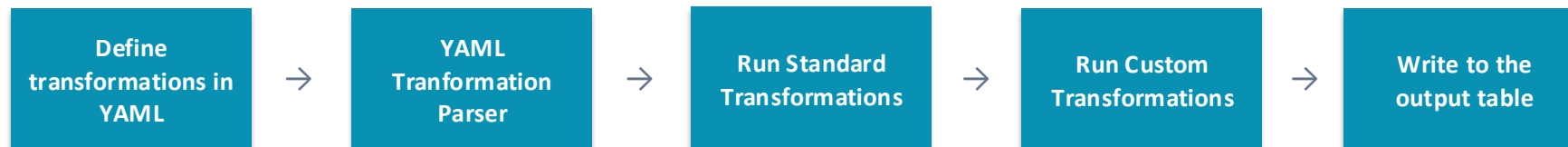
PROBLEM

1. Each Pyspark script in the refined zone correspond to the output table it creates, leading to proliferation of scripts
2. Developers are required to also author unit tests for the corresponding PySpark scripts in the refined zone, ending up spending more time even for simple transformations

APPROACH & OUTCOME

1. Transformations defined in refined zone is often repeatable although require executing in certain order.
2. Config-as-code: A custom tool that parse YAML files, map the required transformations to equivalent Pyspark snippets and write to the output table
3. ~100 scripts files are reduced to mere ~10 (a ~90% reduction)

REFINED ZONE TRANSFORMER WORKFLOW



Evolution 2: Reduce Glue Compute Costs by Running Jobs Locally

Slow jobs are the fastest way to frustrate engineers

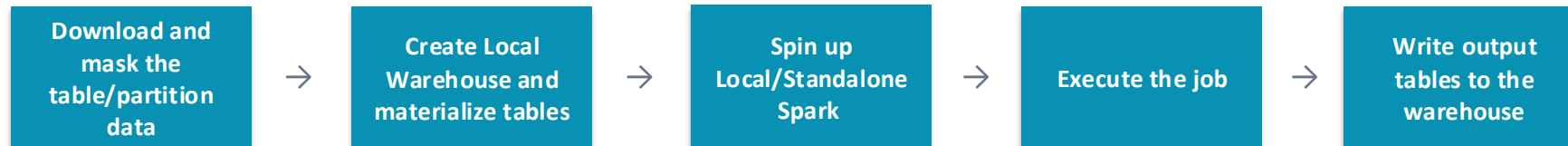
PROBLEM

1. The dev/debug loop to modify Pyspark scripts take 20-30 minutes due to long cluster provision, spin up and execution
2. Cost of running the pipeline 30-50 times increases exponentially.
3. Developer productivity is hit from long feedback loop

APPROACH & OUTCOME

1. Custom developed table downloader and masker tool
2. Local spark is emulated with the same iceberg configurations as Glue
3. Pyspark abstractions to handle both local and Glue jobs
4. Developer productivity is increased manifolds

LOCAL DEBUG MODE WORKFLOW



Evolution 3: Prod deployment using Claude Code Agents

Agents FTW!

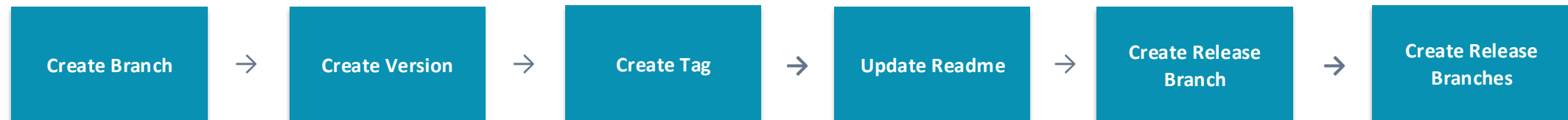
PROBLEM

1. Senior developers often handle production deployments, since failures are costly to fix after release
2. Many steps involved in deployment – Add Readme, Bump up versions, Create Tag for release version, Summarize commits, Create Release Branch, Promote to Staging etc

APPROACH & OUTCOME

1. Using Agents for deployments have made it easy others to participate in this exercise, reducing the reliance on senior developers
2. Agents orchestrate calls to a bunch of bash scripts (to make it deterministic) ,not just the markdown files
3. Everyone in the team can deploy to production without much hesitation

AGENT WORKFLOW FOR PROD DEPLOYMENT



Along the way, we solved many interesting Engineering problems...

- Agent Assisted Production Deployment (Feature/Hotfix/Releases Etc)
- Template based dashboards with design guidelines
- Branching Strategy – GitFlow
- Dev/Stage/Prod Environment Synchronization
- Pulumi to spin up MWAA for environments
- Data Freshness Tracker
- Data Masker
- Data Profilers & Freshness Tracker
- Parity Checkers
- Unit test Suite without Mocks
- Localized testing
- Common Library modules
- Config-driven Development
- Observability and Monitoring

Impact & Result

75%

faster incremental loads

**Zero
Downtime**

schema evolution

Consistent

dashboards across
all consumers

And, happy customers :)



Thank You

Madhurima Saxena & Raghav Nagarajan

Western US Architecture Series

Questions?